

Guia Instrucional - Documentação Viva no BookShelf-API

Atualização contínua de documentação a partir de mudanças reais no código

Material instrucional | Módulo 1 - Semana 07

Objetivo da prática

Configurar um fluxo de documentação viva no projeto BookShelf-API. Ao final, mudanças no código serão usadas como fonte da verdade para orientar a atualização pontual de README, instalação e diagramas, evitando que a documentação seja reescrita sem necessidade.

1. Visão geral

Documentação viva é a prática de manter a documentação alinhada ao comportamento atual do sistema. Em vez de atualizar arquivos manualmente apenas no final do desenvolvimento, a documentação passa a fazer parte do mesmo ciclo de branch, alteração, revisão, teste, commit e Pull Request.

Nesta prática, a documentação será atualizada com apoio de IA, mas a fonte da verdade será o git diff. Isso significa que a IA não deve inventar novas funcionalidades, endpoints ou fluxos. Ela deve analisar apenas o que mudou no código e atualizar somente os documentos realmente impactados.

Conceito	Aplicação nesta prática
Documentação viva	Documentação atualizada junto com o código, dentro do fluxo de entrega.
git diff	Fonte objetiva das mudanças realizadas no projeto.
Prompt template	Arquivo fixo com regras para a IA analisar impacto documental.
Script docs-live	Automação que coleta o diff e gera um prompt final para uso no Kiro ou Copilot.
Revisão humana	Etapa obrigatória para validar se a IA atualizou somente o necessário.

2. Contexto do desafio

O projeto utilizado é o BookShelf-API. O desafio consiste em implementar uma nova regra de negócio na rota de remoção de livros:

Nova regra de negócio

Livros com status reading não podem ser removidos diretamente pela rota DELETE /books/:id.

Essa mudança altera o comportamento da API e, por isso, pode impactar a documentação de uso e os diagramas de fluxo. Porém, ela não altera instalação, dependências, porta, variáveis de ambiente ou comandos de execução. Portanto, nem toda documentação deve ser modificada.

Arquivo	Deve ser atualizado?	Justificativa
README.md	Provavelmente sim	A rota DELETE /books/:id passa a ter uma nova resposta possível: 409 Conflict.
docs/diagrams/api-flow.md	Provavelmente sim	O fluxo da rota passa a ter uma nova condição: livro em leitura.
docs/INSTALLATION.md	Provavelmente não	Não houve mudança de instalação, dependências, comandos, porta ou setup.
openapi.yaml	Não nesta prática	O escopo do desafio restringe a atualização aos documentos definidos no prompt template.

3. Fluxo completo da prática

O fluxo abaixo resume a lógica da atividade. A documentação viva não começa pela edição manual dos arquivos. Ela começa pela alteração no código, seguida pela geração de um prompt baseado no diff.

Etapa	O que será feito	Resultado esperado
1	Clonar o repositório a partir da branch develop	Projeto local pronto para a prática.
2	Criar uma branch para o desafio	Alterações isoladas em uma branch própria.
3	Criar o prompt template	Arquivo docs/prompts/docs-live-update.md.
4	Criar o script docs-live	Script capaz de gerar um prompt a partir do git diff.
5	Implementar a nova regra no DELETE /books/:id	API passa a bloquear remoção de livros em leitura.
6	Gerar o prompt com npm run docs:live	Arquivo .tmp/docs-live/docs-live.generated.md.
7	Usar Kiro ou Copilot para atualizar documentação	Somente documentos impactados são modificados.
8	Revisar, testar e versionar	Código e documentação seguem juntos para o Pull Request.

4. Pré-requisitos

Antes de iniciar, confirme se o ambiente possui as ferramentas necessárias para clonar o projeto, executar comandos Node.js e gerar o prompt automatizado.

- Git configurado com acesso ao GitHub.
- Node.js e npm instalados.
- Python 3 disponível no terminal.
- Acesso ao Kiro, GitHub Copilot ou outro assistente de IA que consiga ler arquivos do projeto.
- Permissão de acesso ao repositório BookShelf-API.

5. Clonar o repositório e preparar a branch

A branch develop será usada como ponto de partida. A partir dela, crie uma branch específica para implementar a regra de negócio e configurar a documentação viva.

Comandos

```
git clone -b develop git@github.com:IA-para-DEVs-SCTEC-T2/bookshelf-api.git
cd bookshelf-api
git branch
git pull origin develop
git checkout -b feature/bookshelf-live-docs
git branch
```

Por que criar uma branch?

A branch isola a mudança e permite que a documentação seja revisada junto com o código no Pull Request. Esse é um ponto central da documentação viva: ela faz parte do ciclo de entrega, não é um arquivo atualizado separadamente depois.

6. Criar a pasta de prompts

A pasta docs/prompts armazenará o prompt template. Esse template contém regras fixas que serão reutilizadas sempre que uma mudança no código precisar ser analisada pela IA.

Comandos

```
mkdir -p docs/prompts
touch docs/prompts/docs-live-update.md
```

O arquivo docs-live-update.md será o modelo base. Mais adiante, o script irá substituir os placeholders `{{CHANGED_FILES}}` e `{{GIT_DIFF}}` pelas mudanças reais detectadas pelo Git.

7. Criar o prompt template

O prompt template define o comportamento esperado da IA. Ele limita quais arquivos podem ser alterados, orienta a análise de impacto e impede que a IA reescreva a documentação inteira sem necessidade.

Regra principal

A IA deve atualizar apenas a documentação impactada pelo diff. Se um documento não for afetado pela mudança, ele deve permanecer inalterado.

Conteúdo de docs/prompts/docs-live-update.md

Documentação viva - atualização por diff

Analise o Git diff abaixo e atualize apenas a documentação impactada.

Arquivos permitidos

Você pode alterar somente:

- `README.md`
- `docs/INSTALLATION.md`
- `docs/diagrams/api-flow.md`

Não altere outros arquivos.

Não crie novos documentos.

Não altere `openapi.yaml`.

Regras gerais

- Use somente mudanças reais presentes no diff.
- Não invente endpoints, comandos, dependências, status codes, mensagens ou fluxos.
- Preserve o estilo atual dos documentos.
- Faça alterações pontuais, sem reescrever arquivos inteiros.
- Se um documento não for impactado, mantenha-o inalterado.
- Ignore contexto anterior e use apenas a seção `Diff detectado`.

Critérios de atualização

Atualize `README.md` quando houver:

- novo endpoint ou alteração em endpoint existente;
- nova funcionalidade visível;
- nova regra de negócio;
- nova validação;
- alteração em status code, mensagem de erro ou exemplo de uso;
- novo comando ou script relevante.

Atualize `docs/INSTALLATION.md` somente quando houver:

- nova dependência;
- alteração em instalação;
- alteração em comando de execução;
- alteração em variável de ambiente;
- alteração de porta;
- alteração no processo de setup.

Não altere `docs/INSTALLATION.md` apenas por nova rota, validação ou regra de negócio.

Atualize `docs/diagrams/api-flow.md` quando houver:

- novo endpoint;
- alteração em rota existente;
- alteração em método HTTP;

- alteração em status code;
- alteração em request, response ou mensagem de erro;
- nova validação ou regra de negócio;
- alteração no fluxo entre cliente, API e resposta.

Diagramas Mermaid

Se a mudança afetar fluxo de API, rota, validação ou regra de negócio, atualize `docs/diagrams/api-flow.md` com um diagrama Mermaid do tipo `sequenceDiagram`.

Para montar o diagrama:

- identifique a rota alterada, como `app.get`, `app.post`, `app.patch`, `app.put` ou `app.delete`;
- identifique o método e o caminho da rota;
- identifique condições reais do código, como `if`, validações e regras de negócio;
- identifique retornos reais, como `return res.status(...).json(...)` ou `return res.status(...).send(...)`;
- crie branches `alt` e `else` somente para condições que aparecem no código;
- use somente status codes e mensagens presentes no diff;
- represente alteração de estado apenas se ela existir no código.

Não inclua automaticamente `400`, `404`, `409`, `200`, `201` ou `204`.

Inclua esses status apenas se aparecerem no diff.

Saída esperada

Atualize diretamente os documentos necessários.

Ao final, informe brevemente:

- mudanças detectadas;
- documentos alterados;
- documentos preservados;
- diagramas criados ou atualizados;
- pontos para revisão humana, se houver.

Arquivos modificados

```
```txt
{{CHANGED_FILES}}
```
```

Diff detectado

```
```diff
{{GIT_DIFF}}
```
```

8. Criar o script de documentação viva

O script docs-live.sh coleta as mudanças do Git, lê o prompt template e gera um novo arquivo de prompt com o diff já preenchido. Esse arquivo final será usado no Kiro ou GitHub Copilot.

Comandos

```
mkdir -p scripts
touch scripts/docs-live.sh
```

Conteúdo de scripts/docs-live.sh

```
#!/usr/bin/env bash
set -e

REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"

PROMPT_TEMPLATE="docs/prompts/docs-live-update.md"
OUTPUT_DIR=".tmp/docs-live"
OUTPUT_PROMPT="$OUTPUT_DIR/docs-live.generated.md"

mkdir -p "$OUTPUT_DIR"

if [ ! -f "$PROMPT_TEMPLATE" ]; then
    echo "Erro: template não encontrado em $PROMPT_TEMPLATE"
    exit 1
fi

python3 <<'PY'
from pathlib import Path
import subprocess
import sys

repo_root = Path.cwd()
template_path = repo_root / "docs/prompts/docs-live-update.md"
output_path = repo_root / ".tmp/docs-live/docs-live.generated.md"

def run_git_command(args):
    result = subprocess.run(
        ["git"] + args,
        cwd=repo_root,
        text=True,
        capture_output=True
    )
    if result.returncode != 0:
        print(result.stderr)
        sys.exit(result.returncode)
    return result.stdout.strip()

working_diff = run_git_command(["diff"])
staged_diff = run_git_command(["diff", "--cached"])
working_files = run_git_command(["diff", "--name-only"])
staged_files = run_git_command(["diff", "--cached", "--name-only"])

all_files = []
for item in (working_files + "
" + staged_files).splitlines():
    item = item.strip()
    if item and item not in all_files:
```

```

all_files.append(item)

git_diff = ""

"".join(part for part in [working_diff, staged_diff] if part.strip())
changed_files = ""
"".join(all_files)

if not git_diff.strip():
    print("Nenhuma mudança detectada pelo Git.")
    print("Altere algum arquivo antes de gerar a documentação viva.")
    sys.exit(0)

content = template_path.read_text(encoding="utf-8")
content = content.replace("{{CHANGED_FILES}}", changed_files)
content = content.replace("{{GIT_DIFF}}", git_diff)

output_path.parent.mkdir(parents=True, exist_ok=True)
output_path.write_text(content, encoding="utf-8")

print()
print("Prompt gerado com sucesso:")
print(output_path)
print()
print("Use este arquivo no Kiro para atualizar a documentação viva.")
PY

```

Permissão de execução

```
chmod +x scripts/docs-live.sh
```

O que esse script resolve?

Ele evita copiar manualmente o git diff para dentro do prompt. A automação reduz erro humano e garante que a IA receba exatamente a mudança que precisa analisar.

9. Ignorar arquivos temporários

A pasta `.tmp/` será usada apenas para armazenar o prompt gerado automaticamente. Ela não deve ser versionada no Git.

Adicionar ao `.gitignore`

```

# .gitignore
.tmp/

```

10. Adicionar o comando no package.json

Para facilitar a execução, adicione um script npm chamado docs:live. Como o comando será executado a partir da pasta backend, o caminho usado abaixo considera que a pasta scripts está na raiz do repositório.

Exemplo em backend/package.json

```
{
  "scripts": {
    "start": "node src/server.js",
    "test": "jest",
    "build": "node -c src/app.js && node -c src/server.js",
    "lint": "eslint src tests",
    "docs:live": "bash ../scripts/docs-live.sh"
  }
}
```

Atenção ao caminho do script

Se o comando docs:live for adicionado ao package.json da raiz do projeto, use bash scripts/docs-live.sh. Se ele for adicionado ao backend/package.json e executado dentro de backend, use bash ../scripts/docs-live.sh.

11. Testar a automação sem mudanças

Antes de alterar a regra de negócio, execute o comando para confirmar que a automação foi configurada corretamente.

Comando

```
cd backend
npm run docs:live
```

Se ainda não houver mudanças no código, a saída esperada será:

Saída esperada

```
Nenhuma mudança detectada pelo Git.
Altere algum arquivo antes de gerar a documentação viva.
```

Objetivo do teste

Validar se o comando docs:live consegue localizar o repositório, encontrar o template e executar o script sem erro.

Por que a execução não gerou prompt?

Porque ainda não existe alteração detectada pelo Git. O script depende de git diff para gerar o prompt final.

12. Implementar a nova regra de negócio

Agora será feita a alteração principal do desafio. Abra o arquivo `src/app.js` e localize a rota DELETE `/books/:id`.

Versão anterior

```
app.delete('/books/:id', (req, res) => {
  const book = findBookById(req.params.id);

  if (!book) {
    return res.status(404).json({
      error: 'Livro não encontrado'
    });
  }

  books = books.filter((item) => item.id !== book.id);
  return res.status(204).send();
});
```

Substitua a rota pela versão abaixo:

Versão atualizada

```
app.delete('/books/:id', (req, res) => {
  const book = findBookById(req.params.id);

  if (!book) {
    return res.status(404).json({
      error: 'Livro não encontrado'
    });
  }

  if (book.status === 'reading') {
    return res.status(409).json({
      error: 'Livro em leitura não pode ser removido diretamente'
    });
  }

  books = books.filter((item) => item.id !== book.id);
  return res.status(204).send();
});
```

| | |
|--------------------------|----------------|
| Livro inexistente | 404 Not Found |
| Livro com status reading | 409 Conflict |
| Livro removível | 204 No Content |

13. Gerar o prompt com base no diff

Depois de alterar o código, gere o prompt final usando o comando configurado. Esse prompt será criado a partir do diff real do projeto.

Comando

```
cd backend  
npm run docs:live
```

O arquivo gerado deve aparecer em:

Arquivo gerado

```
.tmp/docs-live/docs-live.generated.md
```

Esse arquivo contém três partes importantes: instruções do template, lista de arquivos modificados e diff detectado.

14. Conferir o prompt gerado

Antes de usar o prompt na IA, abra o arquivo gerado e confirme se ele contém a mudança correta.

Trecho esperado

```
## Arquivos modificados  
src/app.js  
  
## Diff detectado  
+ if (book.status === 'reading') {  
+   return res.status(409).json({  
+     error: 'Livro em leitura não pode ser removido diretamente'  
+   });  
+ }
```

Critério de validação

Se a seção Diff detectado mostra a nova regra de negócio, o prompt foi gerado corretamente e já pode ser usado no Kiro ou GitHub Copilot.

15. Usar o prompt no Kiro ou GitHub Copilot

O próximo passo é solicitar que a IA leia o prompt gerado e atualize somente os documentos impactados. Use o comando abaixo no assistente de IA do projeto.

Prompt para a IA

Abra e leia novamente o arquivo `.tmp/docs-live/docs-live.generated.md` antes de fazer qualquer alteração.

Use exclusivamente o diff presente na seção "Diff detectado".

Atualize somente a documentação impactada entre:

- README.md
- docs/INSTALLATION.md
- docs/diagrams/api-flow.md

Se a mudança afetar uma rota, validação, regra de negócio ou fluxo da API, atualize `docs/diagrams/api-flow.md` com um diagrama Mermaid derivado dos retornos reais encontrados no diff.

Não altere outros arquivos.

Não crie novos documentos.

Não altere `openapi.yaml` nesta etapa.

Ignore contexto de execuções anteriores.

A IA deve analisar o diff e decidir quais documentos foram impactados.

Por que o prompt restringe arquivos?

Sem restrições, a IA pode modificar arquivos fora do escopo, reescrever documentação inteira ou atualizar partes que não foram impactadas. O objetivo da documentação viva é ser precisa, não expansiva.

16. Entender o impacto esperado da mudança

A nova regra altera o comportamento da rota DELETE `/books/:id`. Portanto, a documentação deve refletir a nova resposta 409 Conflict quando o livro estiver com status reading.

| Documento | Atualização esperada |
|---------------------------|---|
| README.md | Mencionar que DELETE <code>/books/:id</code> retorna 409 Conflict quando o livro está em leitura. |
| docs/diagrams/api-flow.md | Representar o fluxo com 404, 409 e 204 usando Mermaid sequenceDiagram. |
| docs/INSTALLATION.md | Manter inalterado, pois não há impacto de instalação ou execução. |

Um trecho aceitável para o README seria:

Exemplo de texto

- `DELETE /books/:id` - remove um livro por ID. Retorna `409 Conflict` quando o livro está com status `reading`, `404 Not Found` quando o livro não é encontrado e `204 No Content` quando a remoção é concluída com sucesso.

Um exemplo de diagrama esperado para docs/diagrams/api-flow.md:

Exemplo de diagrama Mermaid

```
## Remoção de livro

Este fluxo representa a rota `DELETE /books/:id`.

```mermaid
sequenceDiagram
 participant Cliente
 participant API
 participant Memoria as Memória (books)

 Cliente->>API: DELETE /books/:id
 API->>Memoria: Busca livro pelo ID

 alt Livro não encontrado
 API-->>Cliente: 404 Livro não encontrado
 else Livro em leitura
 API-->>Cliente: 409 Livro em leitura não pode ser removido diretamente
 else Livro removível
 API->>Memoria: Remove livro da lista
 API-->>Cliente: 204 No Content
 end
end
```
```

17. Revisar o que a IA alterou

Depois que a IA atualizar os arquivos, revise o diff manualmente. Essa etapa é obrigatória, porque a IA pode alterar documentos que não deveriam ser modificados ou inventar informações que não existem no código.

Comando de revisão

```
git diff
```

Verifique os seguintes pontos:

- O README menciona a regra do status reading.
- O diagrama representa corretamente as respostas 404, 409 e 204.
- O docs/INSTALLATION.md foi preservado se não houve impacto de setup.
- Nenhum arquivo fora do escopo foi alterado.
- O diagrama Mermaid não inventou status codes ou fluxos inexistentes.

Se algum arquivo foi alterado indevidamente, restaure-o:

Exemplos

```
git restore openapi.yaml  
git restore docs/INSTALLATION.md
```

18. Testar a aplicação

A documentação deve refletir o comportamento real da API. Por isso, depois de alterar o código e a documentação, execute as validações disponíveis no projeto.

Comandos gerais

```
npm install  
npm test  
npm run lint  
npm run build  
npm start
```

Se algum desses comandos não existir no projeto, execute apenas os comandos disponíveis no package.json. O objetivo é garantir que a mudança não quebrou a aplicação.

19. Testar a regra com curl

Com a API em execução, valide os três comportamentos esperados da rota DELETE /books/:id.

Livro com status reading

```
curl -i -X DELETE http://localhost:3000/books/1
```

Resposta esperada:

```
HTTP/1.1 409 Conflict  
  
{  
  "error": "Livro em leitura não pode ser removido diretamente"  
}
```

Livro removível

```
curl -i -X DELETE http://localhost:3000/books/2
```

Resposta esperada:

```
HTTP/1.1 204 No Content
```

Livro inexistente

```
curl -i -X DELETE http://localhost:3000/books/999
```

Resposta esperada:

```
HTTP/1.1 404 Not Found
```

```
{
  "error": "Livro não encontrado"
}
```

Objetivo do teste

Validar se o comportamento documentado corresponde ao comportamento real da API.

Por que o teste pode falhar?

A API pode estar rodando em outra porta, o livro de ID 1 pode não estar com status reading, a rota pode ter sido alterada incorretamente ou o servidor pode não estar ativo.

20. Checklist de coerência da documentação

Antes do commit, confirme se a documentação ficou coerente com o código.

| Item | Critério de aceite |
|---------------------------|--|
| README.md | Descreve DELETE /books/:id com respostas 404, 409 e 204. |
| docs/diagrams/api-flow.md | Mostra o fluxo de remoção com livro inexistente, livro em leitura e livro removível. |
| docs/INSTALLATION.md | Foi preservado, caso não tenha existido impacto em instalação ou execução. |
| openapi.yaml | Não foi alterado nesta etapa. |
| Mermaid | O diagrama renderiza corretamente e não possui fluxos inventados. |
| Código | Testes, lint ou build disponíveis foram executados. |

21. Fazer commit da mudança

Depois de validar código e documentação, versionar tudo junto. O commit deve incluir a alteração de regra de negócio e a documentação impactada.

Comandos

```
git status
git add .
```

```
git commit -m "feat: bloquear a remocao de livros com status reading"
git push origin feature/bookshelf-live-docs
```

Boa prática

Código e documentação devem seguir no mesmo Pull Request. Isso permite que a revisão técnica avalie se o comportamento da API e a documentação estão consistentes.

22. Fluxo final consolidado

A prática completa pode ser resumida como o ciclo abaixo:

| Ordem | Ação |
|-------|--|
| 1 | Criar o prompt template de documentação viva. |
| 2 | Criar o script docs-live.sh. |
| 3 | Configurar o comando npm run docs:live. |
| 4 | Alterar a regra de negócio no código. |
| 5 | Gerar o prompt com o git diff. |
| 6 | Usar IA para atualizar somente a documentação impactada. |
| 7 | Revisar o diff produzido pela IA. |
| 8 | Testar a API. |
| 9 | Commitar código e documentação juntos. |

23. Problemas comuns e como resolver

| Problema | Causa provável | Como resolver |
|---|---|---|
| Nenhuma mudança detectada | Não há git diff no projeto. | Altere um arquivo ou confirme se a mudança foi salva. |
| Template não encontrado | Arquivo docs/prompts/docs-live-update.md não existe ou está em outro caminho. | Confira a pasta docs/prompts e o nome do arquivo. |
| Comando npm falha | Caminho do script incorreto no package.json. | Use bash ../scripts/docs-live.sh se estiver dentro de backend. |
| IA alterou arquivos demais | Prompt sem restrições ou assistente usou contexto anterior. | Reforce o uso exclusivo do diff e restaure arquivos indevidos. |
| INSTALLATION foi alterado sem necessidade | IA interpretou regra de negócio como mudança de setup. | Restaure o arquivo se não houve nova dependência, comando, porta ou variável. |
| Diagrama Mermaid inventou respostas | IA adicionou status codes não presentes no diff. | Remova branches que não aparecem no código. |

24. Aprendizados principais

Ao final desta prática, você deve ser capaz de:

BookShelf-API | Atualizações contínuas guiadas por IA

- Explicar a diferença entre documentação estática e documentação viva.
- Usar git diff como fonte da verdade para orientar a IA.
- Criar um prompt template reutilizável para atualização documental.
- Automatizar a geração de prompts com um script local.
- Identificar quais documentos foram realmente impactados por uma mudança.
- Atualizar README e diagramas com base em uma nova regra de negócio.
- Preservar documentos que não foram impactados.
- Revisar criticamente documentação gerada por IA antes do commit.

Ideia central

Documentação viva não significa atualizar tudo a cada mudança. Significa atualizar somente o que foi impactado, com base no comportamento real do código.

25. Entrega esperada

Ao concluir a prática, o repositório deve conter:

- Branch feature/bookshelf-live-docs criada a partir de develop.
- Arquivo docs/prompts/docs-live-update.md configurado.
- Script scripts/docs-live.sh criado e executável.
- Comando npm run docs:live configurado.
- Regra de negócio implementada em DELETE /books/:id.
- Prompt gerado em .tmp/docs-live/docs-live.generated.md, não versionado.
- README.md atualizado, se impactado.
- docs/diagrams/api-flow.md atualizado com Mermaid, se impactado.
- docs/INSTALLATION.md preservado, se não houver impacto de setup.
- Testes manuais com curl executados para 404, 409 e 204.
- Commit e push realizados para a branch do desafio.